IBM Innovation Centre for Education

NCS Nilson Communication Solutions

RABBI ZIDNI 'ILMA

YENEPOYA
(DEEMED TO BE UNIVERSITY)

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design

On

VISUAL THREAT HUNTING FOR E-COMMERCE NETWORKS

SUBMITTED BY

Muhammed Dhanish - 23BCCDD035

BCA(Cybersecurity, Digital Forensic and Data Science)

UNDER THE GUIDANCE OF

Industry Mentor: Shashank Korrram

Guided by: Suzana Sharol Prabhakar

TABLE OF CONTENTS

1. Introduction 03
2. Scope of the Document 03
3. Intended Audience 04
4. System Overview 04
5. System Design 05
   5.1 Application Design 05
   5.2 Process Flow 06
   5.3 Information Flow 08
   5.4 Components Design 09
   5.5 Key Design Considerations 12
6. API Catalogue 13
7. Data Design 13
   7.1 Data Model 13
   7.2 Data Access Mechanism 14
   7.3 Data Retention Policies 14
   7.4 Data Migration 15
8. Interfaces 15
9. State and Session Management 17
10. Caching 17
11. Non-Functional Requirements 18
12. References 19

1. INTRODUCTION

The **Visual Threat Hunting for E-commerce Networks** project is a cybersecurity analytics system designed to detect, visualize, and report anomalous network traffic patterns in e-commerce web server logs. Built on top of a pre-cleaned dataset of over 10 million Apache access log records from a real Iranian e-commerce platform (**Zanbil, January 2019**), this system applies statistical modeling and interactive dashboarding to enable security analysts to identify potential cyber threats such as DDoS attacks, traffic burst anomalies, and suspicious behavior patterns.

This High Level Design (HLD) document describes the overall system architecture, the analytical components that operate on clean data, the feature engineering strategy, the anomaly detection engine, alert generation logic, and the final Power BI dashboard interface. It defines **WHAT** the system does and **WHY**, while the Low Level Design (LLD) covers the **HOW** of data ingestion and cleaning.

2. SCOPE OF THE DOCUMENT

2.1 In Scope:

- Overall system architecture and component relationships
- Feature engineering design (rolling statistics, Z-score computation, dynamic thresholds)
- Anomaly detection engine: algorithm selection, threshold design, alert classification
- Suspicious window analysis and risk scoring methodology
- Power BI dashboard architecture
- Data export and inter-component data contracts
- Non-functional requirements: performance, security, scalability

2.2 Out of Scope (Covered in LLD):

- Raw log file loading and reading
- Line-by-line regex parsing logic
- Data type conversion mechanics
- Exploratory Data Analysis (EDA) and basic statistical summaries

3. INTENDED AUDIENCE

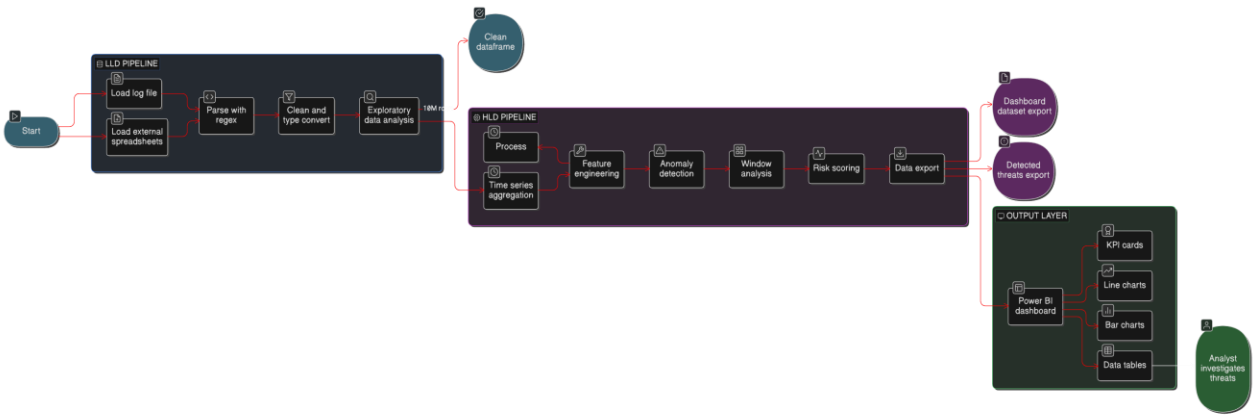
Audience	Purpose
Academic Evaluators	Assess architectural design quality and system thinking
Project Supervisors	Review alignment with stated project objectives
Security Analysts	Understand the threat detection methodology and alert logic
Dashboard Consumers	Understand the Power BI interface and interpret outputs
Future Developers	Extend the detection model or integrate with production systems

4. SYSTEM OVERVIEW

4.1 Project Goal:

- To build a visual threat hunting system that automatically:
- Takes a clean, structured web traffic dataset (output of the LLD pipeline)
- Computes a statistical baseline of normal traffic behavior
- Flags time intervals that deviate significantly from the baseline as anomalies
- Aggregates flagged intervals into 30-minute suspicious windows
- Assigns a quantitative Risk Score to each suspicious window
- Presents all findings through an interactive Power BI dashboard

4.2 System Context Diagram:



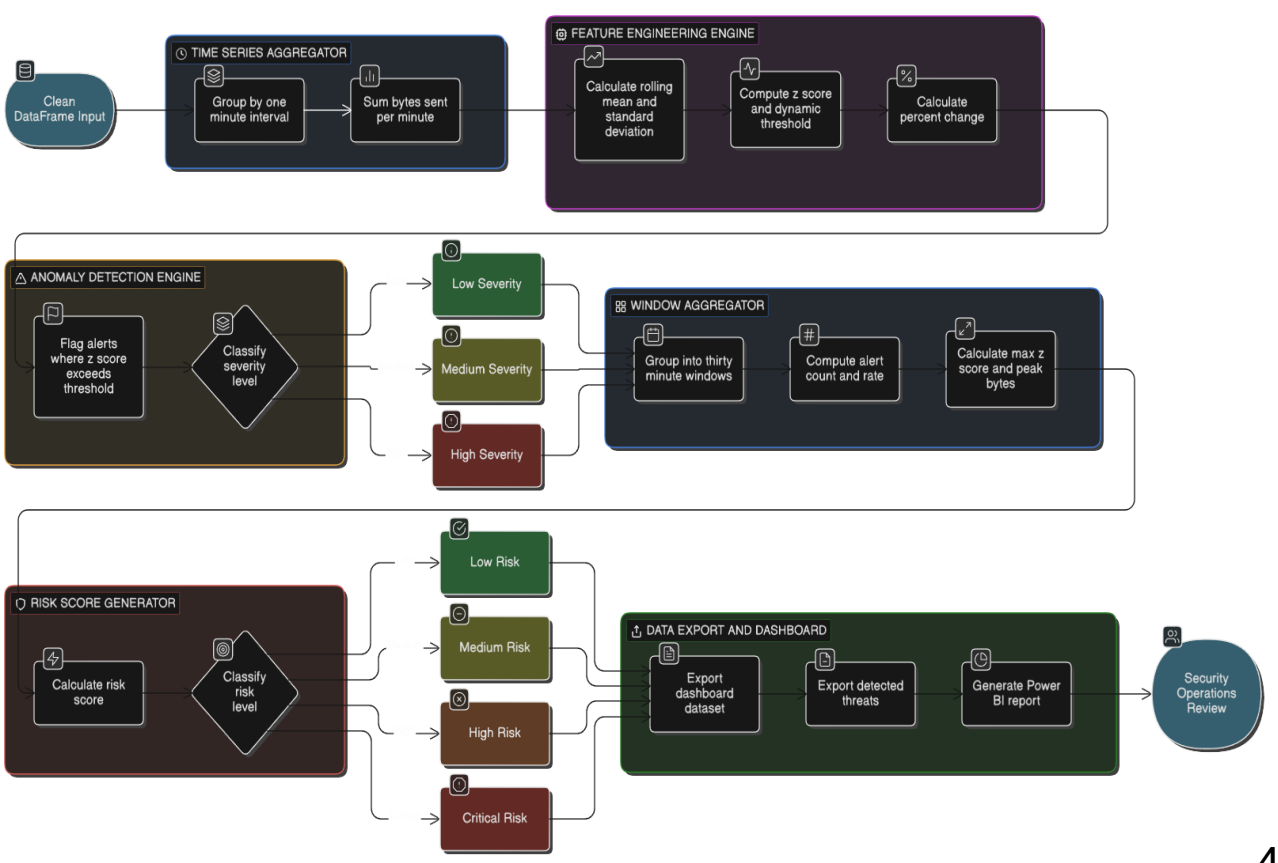
4.3 Key Output Metrics:

Metric	Value
Time intervals analyzed	6,686 (1-minute granularity, ~4.6 days)
Anomaly alerts generated	870 (13.01% alert rate)
Most critical 30-min window	Jan 22, 14:00 — 9 alerts, Z = 2.09
Highest Risk Score	678.99 (Critical severity)
Dashboard pages	3 (Threat Overview, Alert Analytics, Suspicious Windows)

5. SYSTEM DESIGN

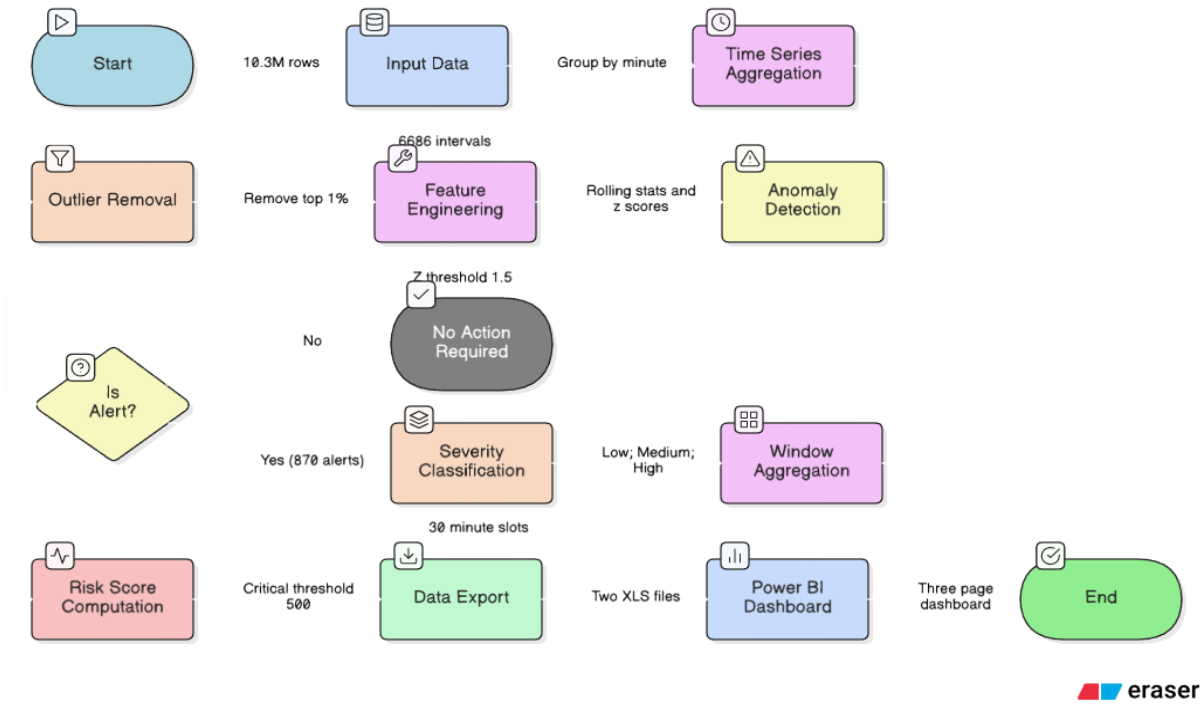
5.1 Application Design:

The HLD pipeline consists of six major components operating sequentially on the clean DataFrame produced by the LLD:



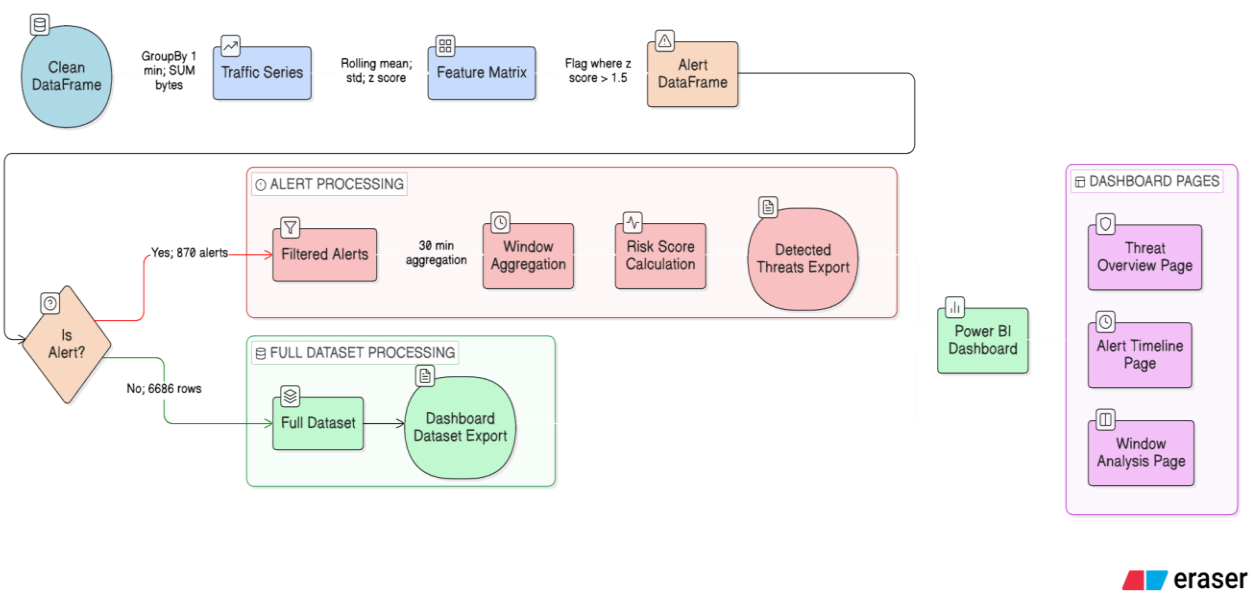
5.2 Process Flow:

The following flowchart shows the complete high-level process from clean data input to final dashboard output.



5.3 Information Flow:

The diagram below shows the data lineage from clean input to dashboard output.



5.4 Components Design:

Component 1: Time-Series Aggregator

Property	Detail
Input	Clean DataFrame: timestamp, bytes columns
Operation	GroupBy 1-minute window, SUM bytes_sent
Key Code	df.groupby(pd.Grouper(key="Time", freq="1min"))["bytes"].sum()
Output	traffic DataFrame: 6,686 rows x [Time, Bytes_Sent]
Rationale	1-minute resolution balances detection granularity with noise

Why 1-minute aggregation? The raw data has ~2,500 individual requests per minute. Individual request-level bytes are too granular for pattern detection. 1-minute aggregation reveals traffic volume waves needed for Z-score baseline modeling, while remaining sensitive enough to detect short DDoS bursts.

Component 2: Feature Engineering Engine

Feature	Formula	Window	Purpose
rolling_mean	Bytes_Sent.rolling(10).mean()	10 min	Adaptive traffic baseline
rolling_std	Bytes_Sent.rolling(10).std()	10 min	Measures normal variability
z_score	(Bytes_Sent - rolling_mean) / rolling_std	—	Standardized deviation
dynamic_threshold	rolling_mean + (3 x rolling_std)	10 min	Upper safe boundary
pct_change	Bytes_Sent.pct_change()	1 min	Captures burst transitions

Why a rolling window of 10 minutes?

Window Size	Problem
3 minutes (too small)	Baseline reacts too fast; each spike inflates mean; next anomaly invisible
10 minutes (chosen)	Balanced: responsive for detection, stable enough not to chase spikes
30 minutes (too large)	Adapts too slowly; sustained 30-min attacks may not trigger alerts

Why Z-score instead of a fixed byte threshold? A fixed threshold (e.g., >50 MB/min = alert) fails because normal peak-hour traffic may naturally exceed it (false positives), and off-peak traffic makes moderate spikes significant (false negatives missed). Z-score normalizes against the current local 10-minute baseline, making detection time-aware and self-calibrating.

Component 3: Anomaly Detection Engine

Property	Detail
Algorithm	Statistical Z-score thresholding
Threshold	Z_THRESHOLD = 1.5 (absolute value of Z-score)
Detection direction	Bidirectional: catches both traffic spikes AND drops
Output	alert boolean column added to traffic DataFrame
Result	870 alerts out of 6,686 intervals = 13.01% alert rate

Z-Threshold Justification:

Threshold	% of Normal Captured	Effect
Z = 1.0	68.3%	Too sensitive — excessive false positives
Z = 1.5 (chosen)	86.6%	Balanced: moderate anomalies captured
Z = 2.0	95.4%	Too lenient — moderate attacks missed
Z = 3.0	99.7%	Extreme events only — most threats missed

Severity Classification:

Severity	Z-Score Range	Alert Count	Share
Low	1.5 <= abs(Z) < 2.0	671	77%
Medium	2.0 <= abs(Z) < 2.5	199	23%
High	abs(Z) >= 2.5	Rare	<1%

Component 4: Suspicious Window Aggregator

Purpose: Group per-minute alerts into 30-minute threat windows to distinguish sustained attack campaigns from isolated random spikes.

Property	Detail
Window size	30 minutes
Grouping method	pd.Grouper(key="Time", freq="30min")
Metrics per window	Alert_Count, Total_Intervals, Alert_Rate_%, Max_Abs_Z, Peak_Bytes
Sort order	Alert_Count DESC, then Max_Abs_Z DESC

Top 10 Suspicious 30-Minute Windows:

Rank	Window Start	Alerts	Alert Rate	Max Z
1	2019-01-22 14:00	9	30.00%	2.09
2	2019-01-23 06:30	7	23.33%	2.38
3	2019-01-26 20:00	7	23.33%	2.23
4	2019-01-23 17:00	7	23.33%	2.22
5	2019-01-25 11:30	7	23.33%	2.11
6	2019-01-24 14:00	7	23.33%	1.86
7	2019-01-25 06:00	7	23.33%	1.82
8	2019-01-25 02:00	6	20.00%	2.46
9	2019-01-26 08:30	6	20.00%	2.34
10	2019-01-26 17:30	6	20.00%	2.25

Component 5: Risk Score Generator

Risk Score Formula:

Risk_Score = Alert_Count x Max_Abs_Z x log10(Peak_Bytes + 1)

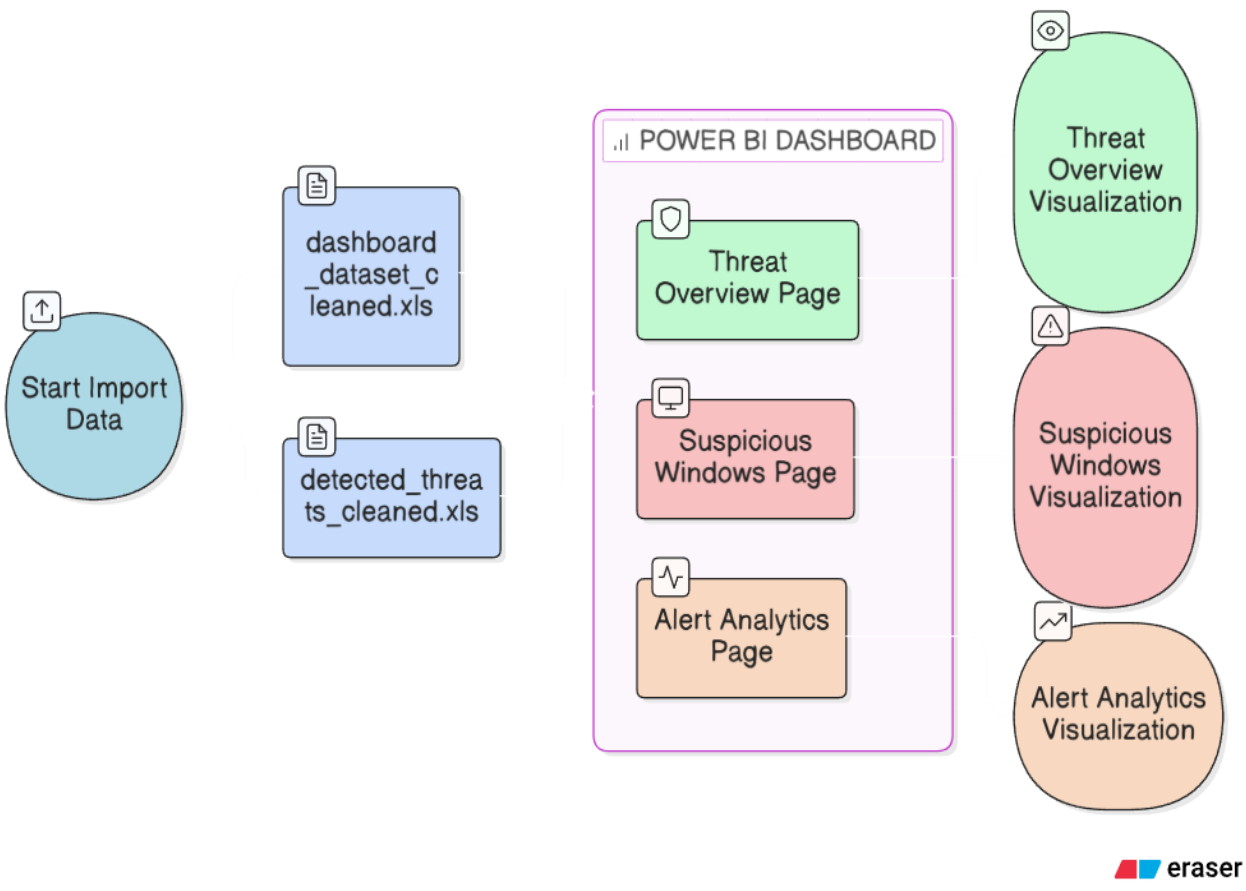
Formula design rationale: **Alert_Count** captures sustained patterns; **Max_Abs_Z** captures severity of deviation; **log10(Peak_Bytes+1)** captures traffic volume while dampening extreme outliers.

Band	Score Range	Required Action
Low	0 — 100	Log and monitor
Medium	100 — 300	Assign for investigation
High	300 — 500	Active response required
Critical	> 500	Immediate incident response

Maximum observed: **678.99** -> **CRITICAL** (Jan 22, 14:00 window)

Component 6: Power BI Dashboard:

The dashboard is a 3-page interactive report providing a comprehensive threat landscape view.



5.5 Key Design Considerations:

5.5.1 Multi-Resolution Time Strategy

Resolution	Used For	Rationale
1-minute intervals	Alert detection	Fine-grained: catches short DDoS bursts
10-minute rolling win	Baseline computation	Balances noise smoothing vs responsiveness
30-minute windows	Threat reporting	Groups related alerts into analyst-friendly periods

A single anomalous minute could be random noise. Multiple anomalous minutes within a 30-minute window strongly indicate a sustained attack pattern.

5.5.2 Self-Adapting Baseline:

The rolling mean automatically adjusts to:

Time-of-day patterns (peak hours vs overnight lulls)

Day-of-week traffic cycles

Long-term gradual traffic growth

This makes the system **zero-configuration**: no manual threshold updates are needed.

5.5.3 Algorithm Selection Justification:

Approach	Advantage	Disadvantage
Fixed threshold	Simple to implement	Fails across different traffic load levels
Z-score (chosen)	Self-adapting, interpretable, no training data needed	Assumes local normality
Isolation Forest (ML)	Learns complex multi-dimensional patterns	Requires labelled training data
LSTM (Deep Learning)	Captures long-range temporal dependencies	Needs large labelled dataset; black-box

6. API CATALOGUE

This system is a self-contained batch analytics pipeline with no REST or HTTP endpoints. The following documents all functional interfaces between internal components:

Interface	From	To	Data Exchanged	Format
Clean DataFrame handoff	LLD Pipeline	Time-Series Aggregator	10.3M row DataFrame	pandas in-memory
Traffic series	Time-Series Agg.	Feature Engineer	6,686 x 2 DataFrame	pandas in-memory
Feature matrix	Feature Engineer	Anomaly Detector	6,686 x 7 DataFrame	pandas in-memory
Alert DataFrame	Anomaly Detector	Window Aggregator	6,686 x 8 (with alert)	pandas in-memory
Window DataFrame	Window Aggregator	Risk Scorer	~223 x 5 DataFrame	pandas in-memory
Export to files	Risk Scorer	File System	2 XLS output files	.xls Excel format
Dashboard import	File System	Power BI	2 structured datasets	Excel file import

7. DATA DESIGN

7.1 Data Model:

7.1.1 Feature-Engineered Traffic DataFrame (Central HLD Data Structure)

Column	Type	Description
Time	datetime	Start of 1-minute interval (UTC+03:30)
Bytes_Sent	int64	Total bytes transmitted in this minute
rolling_mean	float64	Mean of Bytes_Sent over previous 10 minutes
rolling_std	float64	Std dev of Bytes_Sent over previous 10 minutes
z_score	float64	(Bytes_Sent - rolling_mean) / rolling_std
dynamic_threshold	float64	rolling_mean + 3 x rolling_std
pct_change	float64	% change in Bytes_Sent from previous minute
alert	bool	True if abs(z_score) > 1.5

7.1.2 Suspicious Window DataFrame:

Column	Type	Description
Time	datetime	Start of the 30-minute window
Alert_Count	int64	Number of alert=True 1-min intervals in window
Total_Intervals	int64	Total 1-min intervals in window (typically 30)
Alert_Rate_%	float64	Alert_Count / Total_Intervals x 100
Max_Abs_Z	float64	Maximum absolute Z-score in the window
Peak_Bytes	int64	Maximum Bytes_Sent value in the window
Risk_Score	float64	Alert_Count x Max_Abs_Z x log10(Peak_Bytes+1)

7.1.3 Exported File Schemas:

dashboard_dataset_cleaned.xls — Rows: 6,686 (all 1-minute traffic intervals). Columns: All 8 from Feature-Engineered DataFrame. Power BI Consumer: Pages 1 and 2.

detected_threats_cleaned.xls — Rows: 870 (rows where alert == True only). Columns: Same 8 columns, filtered subset. Power BI Consumer: Pages 2 and 3.

7.2 Data Access Mechanism:

Data Store	Access Method	Access Pattern
Clean DataFrame (from LLD)	pandas in-memory	Sequential, all-at-once
Intermediate DataFrames	pandas in-memory	Created and consumed within session
dashboard_dataset.xls	df.to_excel() + Power BI import	Write once, read once
detected_threats.xls	df.to_excel() + Power BI import	Write once, read once
Power BI data model	Internal Power BI relationships	Time-based join on Time column

7.4 Data Migration:

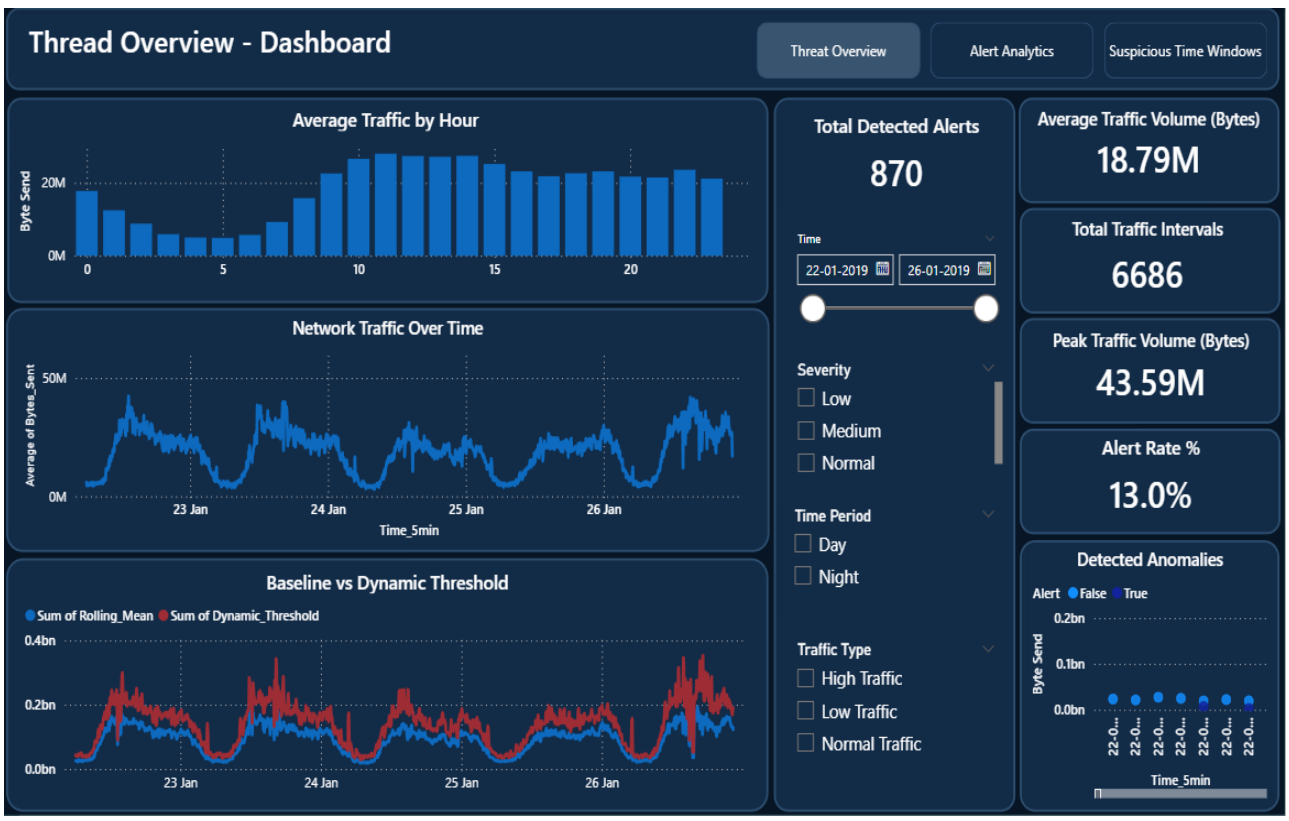
The current system processes a static historical dataset. For future production deployment:

Current (Batch Mode)	Future (Streaming Mode)
Load full log file at once	Consume from Kafka/syslog stream
pandas groupby(freq='1min')	Spark Structured Streaming 1-min tumbling windows
Rolling window on DataFrame	Stateful streaming aggregation functions
df.to_excel() export	Write to database or cloud data lake
Power BI static XLS import	Power BI DirectQuery or live JDBC connection

8. INTERFACES

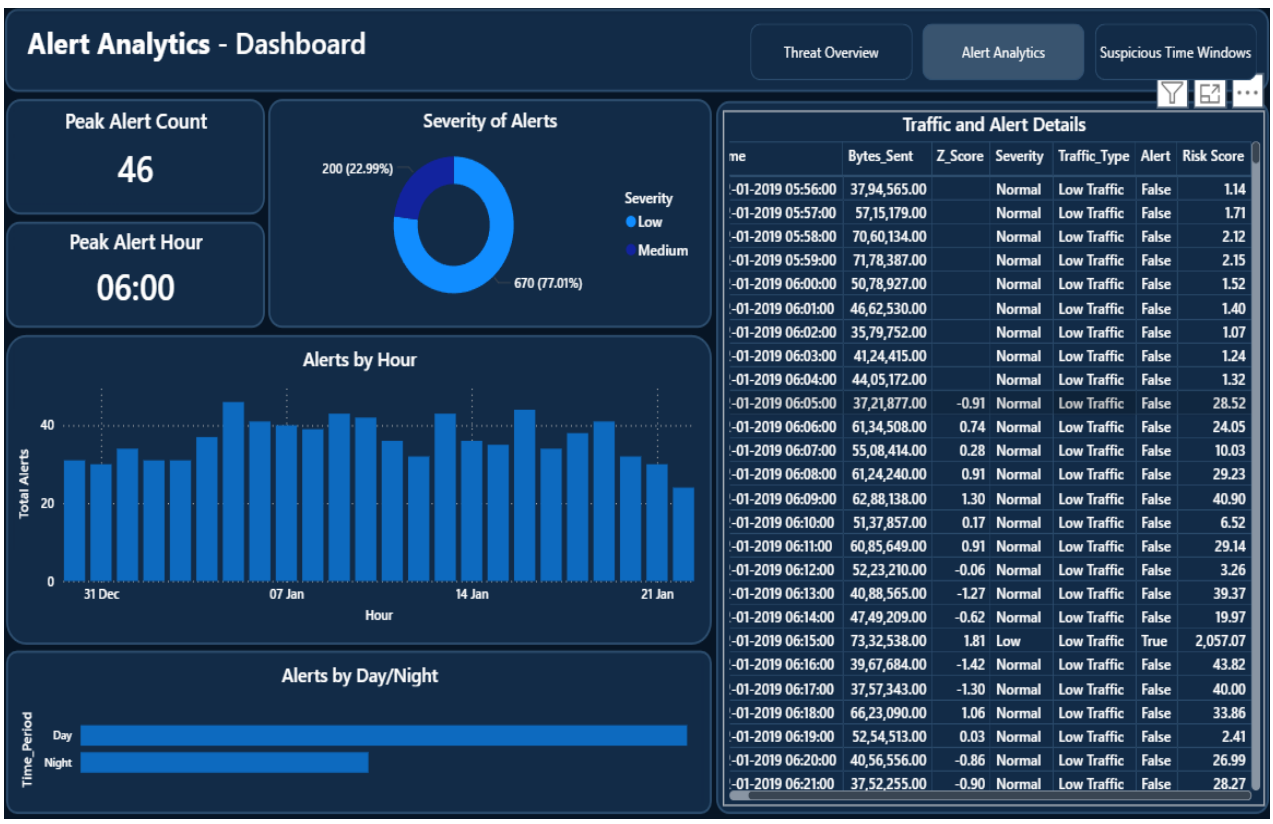
8.1 Page 1: Threat Overview:

Provides a complete 4-day threat landscape summary for security leadership.



8.2 Page 2: Alert Analytics:

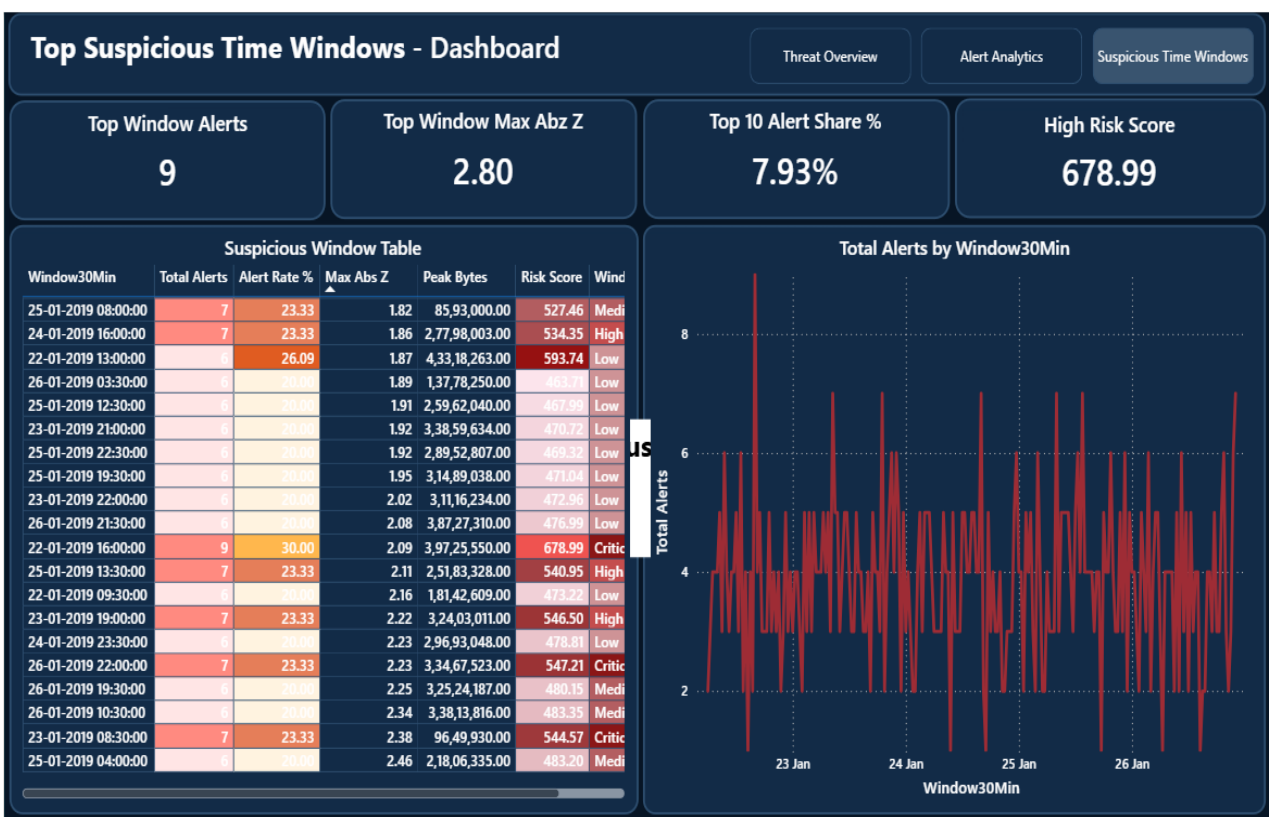
Drills into the 870 detected alerts with breakdowns for SOC analysts.



me	Bytes_Sent	Z_Score	Severity	Traffic_Type	Alert	Risk Score
-01-2019 05:56:00	37,94,565.00		Normal	Low Traffic	False	1.14
-01-2019 05:57:00	57,15,179.00		Normal	Low Traffic	False	1.71
-01-2019 05:58:00	70,60,134.00		Normal	Low Traffic	False	2.12
-01-2019 05:59:00	71,78,387.00		Normal	Low Traffic	False	2.15
-01-2019 06:00:00	50,78,927.00		Normal	Low Traffic	False	1.52
-01-2019 06:01:00	46,62,530.00		Normal	Low Traffic	False	1.40
-01-2019 06:02:00	35,79,752.00		Normal	Low Traffic	False	1.07
-01-2019 06:03:00	41,24,415.00		Normal	Low Traffic	False	1.24
-01-2019 06:04:00	44,05,172.00		Normal	Low Traffic	False	1.32
-01-2019 06:05:00	37,21,877.00	-0.91	Normal	Low Traffic	False	28.52
-01-2019 06:06:00	61,34,508.00	0.74	Normal	Low Traffic	False	24.05
-01-2019 06:07:00	55,08,414.00	0.28	Normal	Low Traffic	False	10.03
-01-2019 06:08:00	61,24,240.00	0.91	Normal	Low Traffic	False	29.23
-01-2019 06:09:00	62,88,138.00	1.30	Normal	Low Traffic	False	40.90
-01-2019 06:10:00	51,37,857.00	0.17	Normal	Low Traffic	False	6.52
-01-2019 06:11:00	60,85,649.00	0.91	Normal	Low Traffic	False	29.14
-01-2019 06:12:00	52,23,210.00	-0.06	Normal	Low Traffic	False	3.26
-01-2019 06:13:00	40,88,565.00	-1.27	Normal	Low Traffic	False	39.37
-01-2019 06:14:00	47,49,209.00	-0.62	Normal	Low Traffic	False	19.97
-01-2019 06:15:00	73,32,538.00	1.81	Low	Low Traffic	True	2,057.07
-01-2019 06:16:00	39,67,684.00	-1.42	Normal	Low Traffic	False	43.82
-01-2019 06:17:00	37,57,343.00	-1.30	Normal	Low Traffic	False	40.00
-01-2019 06:18:00	66,23,090.00	1.06	Normal	Low Traffic	False	33.86
-01-2019 06:19:00	52,54,513.00	0.03	Normal	Low Traffic	False	2.41
-01-2019 06:20:00	40,56,556.00	-0.86	Normal	Low Traffic	False	26.99
-01-2019 06:21:00	37,52,255.00	-0.90	Normal	Low Traffic	False	28.27

8.3 Page 3: Suspicious Time Windows:

Focuses on 30-minute aggregated threat windows and Risk Scores for incident prioritization.



Window30Min	Total Alerts	Alert Rate %	Max Abs Z	Peak Bytes	Risk Score	Wind
25-01-2019 08:00:00	7	23.33	1.82	85,93,000.00	527.46	Medi
24-01-2019 16:00:00	7	23.33	1.86	2,77,98,003.00	534.35	High
22-01-2019 13:00:00	8	26.09	1.87	4,33,18,263.00	593.74	Low
26-01-2019 03:30:00	6	20.00	1.89	1,37,78,250.00	463.71	Low
25-01-2019 12:30:00	6	20.00	1.91	2,59,62,040.00	467.99	Low
23-01-2019 21:00:00	6	20.00	1.92	3,38,59,634.00	470.72	Low
25-01-2019 22:30:00	6	20.00	1.92	2,89,52,807.00	469.32	Low
25-01-2019 19:30:00	6	20.00	1.95	3,14,89,038.00	471.04	Low
23-01-2019 22:00:00	6	20.00	2.02	3,11,16,234.00	472.96	Low
26-01-2019 21:30:00	6	20.00	2.08	3,87,27,310.00	476.99	Low
22-01-2019 16:00:00	9	30.00	2.09	3,97,25,550.00	678.99	Critic
25-01-2019 13:30:00	7	23.33	2.11	2,51,83,328.00	540.95	High
22-01-2019 09:30:00	6	20.00	2.16	1,81,42,609.00	473.22	Low
23-01-2019 19:00:00	7	23.33	2.22	3,24,03,011.00	546.50	High
24-01-2019 23:30:00	6	20.00	2.23	2,96,93,048.00	478.81	Low
26-01-2019 22:00:00	7	23.33	2.23	3,34,67,523.00	547.21	Critic
26-01-2019 19:30:00	6	20.00	2.25	3,25,24,187.00	480.15	Medi
26-01-2019 10:30:00	6	20.00	2.34	3,38,13,816.00	483.35	Medi
23-01-2019 08:30:00	7	23.33	2.38	96,49,930.00	544.57	Critic
25-01-2019 04:00:00	6	20.00	2.46	2,18,06,335.00	483.20	Medi

9. STATE AND SESSION MANAGEMENT

The system is a **stateless**, single-session batch pipeline with no ongoing state between runs.

Property	Value
Session type	Single Jupyter Notebook kernel execution
State persistence	None — all variables exist in RAM during session only
User sessions	Not applicable — single analyst operates the notebook
Authentication	Not applicable — local file-based execution only
Re-run behavior	Full pipeline re-executes from start; output files are overwritten

The Power BI dashboard is a **static, read-only** interface. It imports data once from the exported XLS files and holds no live connection to the Jupyter Notebook or raw log file.

11. NON-FUNCTIONAL REQUIREMENTS

11.1 Security Aspects:

Aspect	Current State	Recommendation
Data Sensitivity	Logs contain real user IPs and browsing behavior	Anonymize IPs (hash or mask last octet)
Access Control	Log file accessible to all OS users on machine	Restrict to authorized analysts only
Dashboard Sharing	Power BI .pbix embeds data internally	Use Power BI Service workspace with RBAC
Code Security	No credentials or secrets in notebook	Maintain this — use env variables if auth added
Output Sensitivity	XLS exports contain IP/traffic data	Apply same policies as source data
Audit Logging	No audit trail of analysis runs	Add execution timestamp + analyst-ID logging

11.2 Performance Aspects:

Metric	Current Value	Notes
Input data (from LLD)	10,363,637 rows (~474 MB)	Received as clean pandas DataFrame
Feature engineering time	1 — 5 seconds	Fully vectorized pandas rolling operations
Anomaly detection time	Less than 1 second	Single column boolean comparison
Window aggregation time	1 — 5 seconds	pandas groupby with 4 agg. functions
Excel export time	10 — 30 seconds per file	XLS slower than CSV; needed for Power BI
Total HLD pipeline time	1 — 3 minutes	Dominated by Excel export stage
Power BI import time	5 — 15 seconds	Static file import, no live query
Dashboard interaction time	Less than 2 seconds	Pre-computed static data, fast filter response
Scalability ceiling	~50M rows on 32 GB RAM	Beyond this: migrate to Apache Spark or Dask

12. REFERENCES

1. **Dataset:** Zanbil E-commerce Apache Access Logs, Kaggle (January 2019) — https://www.kaggle.com/datasets/eliasdabbas/web-server-access-logs
2. **Z-Score Anomaly Detection:** Shewhart, W.A. (1931). Economic Control of Quality of Manufactured Product. — Statistical process control: foundational work on Z-score thresholding.
3. **Rolling Window Time Series:** Box, G.E.P. and Jenkins, G.M. (1976). Time Series Analysis: Forecasting and Control. — Holden-Day.
4. **DDoS Detection Research:** Bhuyan, M.H., et al. (2014). Network Anomaly Detection: Methods, Systems and Tools. — IEEE Communications Surveys and Tutorials, 16(1), pp. 303-336.
5. **pandas Rolling Window Documentation:** pandas Development Team — https://pandas.pydata.org/docs/reference/api/pandas.core.window.rolling.Rolling.mean.html
6. **Power BI Documentation:** Microsoft Corporation — https://learn.microsoft.com/en-us/power-bi/
7. **Apache Access Log Format:** Apache HTTP Server Project — https://httpd.apache.org/docs/current/logs.html
8. **IBM HLD Template Format:** Provided by Industry Partner / University